



INSTITUTO FEDERAL DO PIAUÍ
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Kamila Rocha

Mylena Duarte

Testes de Técnica de Caixa Branca

TERESINA-PIAUÍ

09/2026

1. Código selecionado e regra de negócio

Projeto: Controle de Medicação na Internação - disciplina de Programação para Internet II

Linguagem: Typescript

Função principal:

```
1  function validateMedicationInput(body: any) {
2      if (isBlank(body.patientName)) {
3          return "O campo Nome do paciente é obrigatório";
4      }
5
6      if (isBlank(body.medicationName)) {
7          return "O campo Nome do medicamento é obrigatório";
8      }
9
10     if (isBlank(body.dosage)) {
11         return "O campo Dosagem é obrigatória";
12     }
13
14     if (isBlank(body.route)) {
15         return "O campo Via de administração é obrigatória";
16     }
17
18     if (isBlank(body.scheduledAt) || !ISO_DATE_TIME.test(body.scheduledAt)) {
19         return "O campo Data e horário são obrigatórios e deve estar no formato AAAA-MM-DD.";
20     }
21
22     return null;
23 }
```

Função auxiliar:

```
1  function isBlank(value: unknown): boolean {
2      return typeof value !== "string" || value.trim() === "";
3  }
```

Regra de negócio: A função `validateMedicationInput` valida os dados que são obrigatórios para cadastro de uma medicação. Ela verifica se os campos nome do paciente, dosagem, via de administração e data/horário foram preenchidos corretamente. Quando encontra um valor inválido, retorna uma mensagem de erro específica. Caso todos os dados sejam válidos, retorna `null`.

Entradas, saída esperada e exceções/retornos antecipados:

- Entradas: o objeto `body`, com os campos `patientName`, `medicationName`, `dosage`, `route` e `scheduledAt`
- Saída esperada: `null` quando todos os campos são válidos (dados aceitos)

- Retornos antecipados (exceções de negócio): uma string de erro específica para cada campo inválido — são 5 possíveis mensagens diferentes, uma por decisão (N3 a N8)

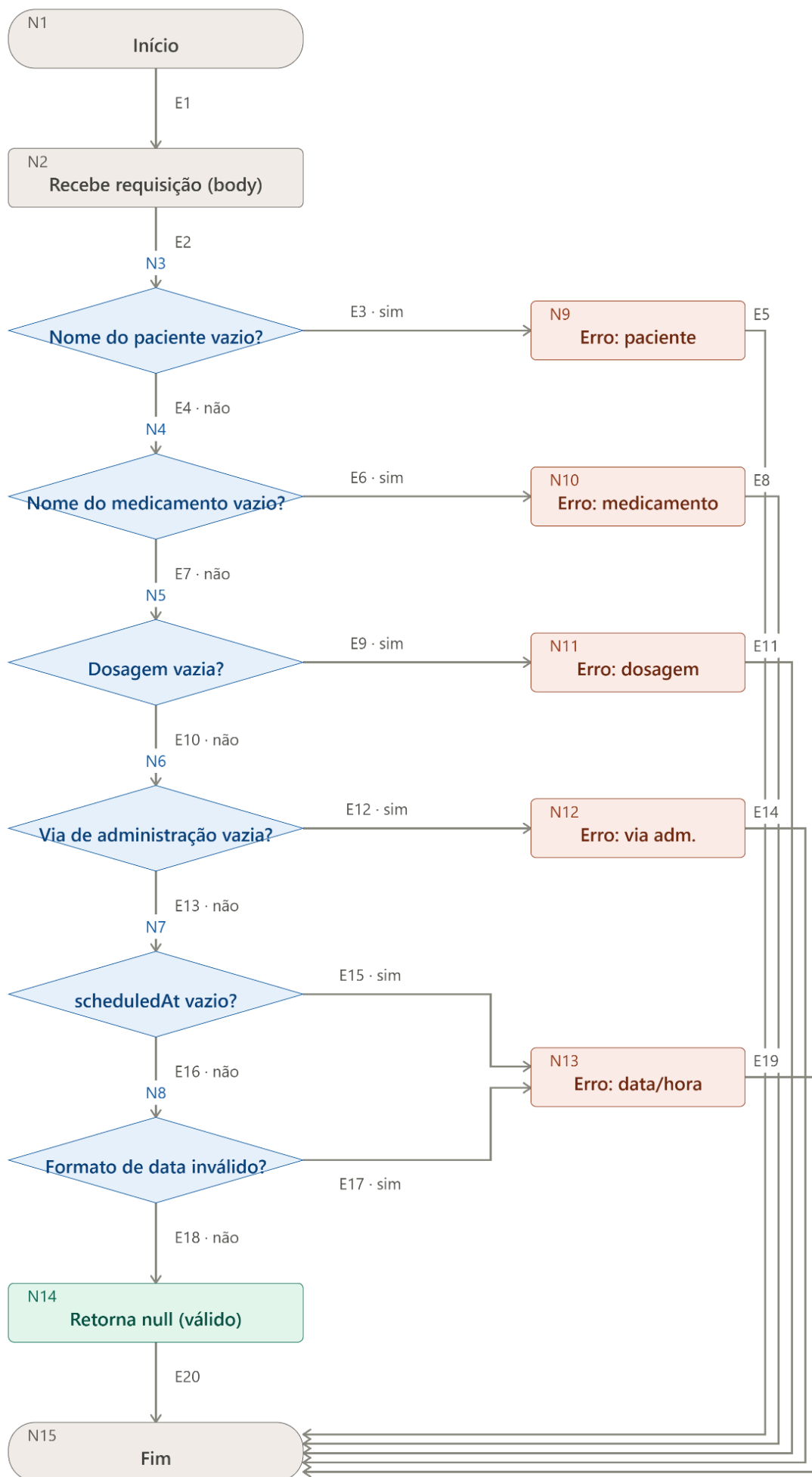
2. Grafo de Fluxo de Controle

O Grafo de Fluxo de Controle apresentado, inicia seu fluxo no nó N1 e, após o recebimento da requisição em N2, passa **sequencialmente** pelas verificações dos campos de nome do paciente, nome do medicamento, dosagem, via de administração, data e horário.

Cada validação é representada por um nó de decisão, possuindo uma saída verdadeira, direcionada ao respectivo nó de erro, e uma saída falsa, que permite a continuidade da validação para o próximo campo. Dessa forma, quando algum campo apresenta erro, ocorre um retorno antecipado, direcionando o fluxo para o nó correspondente à mensagem de erro e, posteriormente, para o nó N15 (Fim).

Para o campo `scheduledAt`, o fluxo apresenta duas verificações: primeiro é analisado se o campo está vazio (N7) e, caso não esteja, é verificado se seu formato é válido (N8). Por ser uma **condição composta**, foi feita a decomposição dela em dois nós para que fosse possível representar o teste de ambos os caminhos. Quando ambas as validações são superadas, o fluxo alcança N14, que representa o retorno null, indicando que os dados são válidos.

O grafo possui 15 nós ($N = 15$) e 20 arestas ($E = 20$), formando um único componente conectado ($P = 1$). Além disso, são identificados 6 pontos de decisão ($D = 6$). Esses elementos serão utilizados posteriormente para o cálculo da complexidade ciclomática, bem como para a definição dos caminhos linearmente independentes e dos casos de teste correspondentes.



2.1 Significado das saídas

Abaixo, é apresentado o significado de cada saída das instruções de decisão.

Nó	Decisão	V	F
N3	<code>isBlank(body.patientName)</code>	Paciente está em branco → retorna erro	Paciente preenchido → continua
N4	<code>isBlank(body.medicationName)</code>	Medicamento está em branco → retorna erro	Medicamento preenchido → continua
N5	<code>isBlank(body.dosage)</code>	Dosagem está em branco → retorna erro	Dosagem preenchida → continua
N6	<code>isBlank(body.route)</code>	Via está em branco → retorna erro	Via preenchida → continua
N7	<code>isBlank(body.scheduledAt)</code>	Data/horário está em branco → retorna erro	Data/horário preenchido → verifica formato
N8	<code>!ISO_DATE_TIME.test(body.scheduledAt)</code>	Formato inválido → retorna erro	Formato válido → retorna <code>null</code>

2.2. Correspondência nó × código

A numeração apresentada no código estabelece a correspondência entre cada instrução ou bloco de instruções e o respectivo nó representado no Grafo de Fluxo de Controle. Essa identificação permite relacionar diretamente o fluxo apresentado no grafo com as instruções responsáveis por cada etapa da execução da função.

```
1  // N3
2  if (isBlank(body.patientName)) {
3  // N9
4      return "O campo Nome do paciente é obrigatório";
5  }
6
7  // N4
8  if (isBlank(body.medicationName)) {
9  // N10
10     return "O campo Nome do medicamento é obrigatório";
11 }
12
13 // N5
14 if (isBlank(body.dosage)) {
15 // N11
16     return "O campo Dosagem é obrigatória";
17 }
18
19 // N6
20 if (isBlank(body.route)) {
21 // N12
22     return "O campo Via de administração é obrigatória";
23 }
24
25 // N7 · N8
26 if (isBlank(body.scheduledAt) || !ISO_DATE_TIME.test(body.scheduledAt)) {
27 // N13
28     return "O campo Data e horário são obrigatórios e deve estar no formato AAAA-MM-DD.";
29 }
30
31 // N14
32 return null;
33 // N15
34 }
```

3. Complexidade ciclomática

Como visto no tópico anterior, para o GFC da função `validateMedicationInput()`, foram identificados 15 nós (N), 20 arestas (E), 1 componente conectado (P) e 6 pontos de decisão (D).

Pela fórmula baseada em nós e arestas:

$$V(G) = E - N + 2P$$

$$V(G) = 20 - 15 + 2(1)$$

$$V(G) = 7$$

Pela fórmula baseada nos pontos de decisão:

$$V(G) = D + 1$$

$$V(G) = 6 + 1$$

$$V(G) = 7$$

Também foi realizada a verificação da complexidade ciclomática utilizando a ferramenta ESLint, destinada à análise de código JavaScript. O resultado obtido pela ferramenta foi $V(G) = 7$, coincidindo com os valores calculados manualmente pelas duas fórmulas.

Portanto, a complexidade ciclomática da função é 7.

Como a complexidade ciclomática obtida é 7, conclui-se que são necessários inicialmente 7 caminhos básicos e, conseqüentemente, 7 casos de teste para exercitar esses caminhos.

4. Critérios baseados no fluxo de controle

ID	Entrada/cenário	Resultado esperado	Caminho pelas decisões	Critério atendido
CT-01	patientName vazio	Retorna erro informando que o nome do paciente é obrigatório	$N3 = V$	Nós, arestas e caminho
CT-02	patientName preenchido e medicationName vazio	Retorna erro informando que o nome do medicamento é obrigatório	$N3 = F \rightarrow N4 = V$	Nós, arestas e caminho
CT-03	Paciente e medicamento preenchidos; dosage vazio	Retorna erro informando que a dosagem é obrigatória	$N3 = F \rightarrow N4 = F \rightarrow N5 = V$	Nós, arestas e caminho
CT-04	Campos anteriores preenchidos; route vazio	Retorna erro informando que a via de administração é obrigatória	$N3 = F \rightarrow N4 = F \rightarrow N5 = F \rightarrow N6 = V$	Nós, arestas e caminho
CT-05	Campos anteriores válidos; scheduledAt vazio	Retorna erro relacionado à data/horário	$N3 = F \rightarrow N4 = F \rightarrow N5 = F \rightarrow N6 = F \rightarrow N7 = V$	Nós, arestas, condição composta e caminho
CT-06	Campos anteriores válidos; scheduledAt preenchido, porém em formato inválido	Retorna erro relacionado à data/horário	$N3 = F \rightarrow N4 = F \rightarrow N5 = F \rightarrow N6 = F \rightarrow N7 = F \rightarrow N8 = V$	Nós, arestas, condição composta e caminho
CT-07	Todos os campos preenchidos corretamente e scheduledAt em formato válido	Retorna null	$N3 = F \rightarrow N4 = F \rightarrow N5 = F \rightarrow N6 = F \rightarrow N7 = F \rightarrow N8 = F$	Nós, arestas, condição composta e caminho

5. Critério baseado na complexidade: caminhos básicos

Como $V(G) = 7$, foram montados 7 caminhos independentes (N3 a N8), um por decisão do grafo. O CB-01 é o erro mais simples (paciente vazio), e os seguintes vão avançando uma decisão a mais, até o CB-07, que passa por todas como falsas e retorna null.

Caminho	Sequência de nós	Nova aresta introduzida	Caso de teste associado
CB-01	N1-N2-N3-N9-N15	N3→N9 (V)	CT-01
CB-02	N1-N2-N3-N4-N10-N15	N4→N10 (V)	CT-02
CB-03	N1-N2-N3-N4-N5-N11-N15	N5→N11 (V)	CT-03
CB-04	N1-N2-N3-N4-N5-N6-N12-N15	N6→N12 (V)	CT-04
CB-05	N1-N2-N3-N4-N5-N6-N7-N13-N15	N7→N13 (V)	CT-05
CB-06	N1-N2-N3-N4-N5-N6-N7-N8-N13-N15	N7→N8 (F) e N8→N13 (V)	CT-06
CB-07	N1-N2-N3-N4-N5-N6-N7-N8-N14-N15	N8→N14 (F)	CT-07

O CB-05 e o CB-06 são parecidos, mas diferentes: como a condição do `scheduledAt` usa OU, ela foi dividida em duas decisões (N7 e N8). No CB-05, o campo já está vazio e o fluxo para em N7. No CB-06, o campo está preenchido, mas com formato errado, então passa por N7 e para em N8.

6. Implementação e execução dos testes

CT01: Verifica a rejeição de *patientName* vazio, exercitando a saída verdadeira da primeira decisão.

```
test("CT01 - rejeita paciente em branco", () => {
  const input = {
    patientName: "",
    medicationName: "Dipirona",
    dosage: "500mg",
    route: "oral",
    scheduledAt: "2026-09-03T10:00"
  };

  const result = validateMedicationInput(input);

  expect(result).toBe("O campo Nome do paciente é obrigatório");
});
```

CT02: Verifica a rejeição de *medicationName* vazio após a validação de *patientName*. Atende a nós, arestas e caminho.

```
test("CT02 - rejeita medicamento em branco", () => {
  const input = {
    patientName: "Jorge",
    medicationName: "",
    dosage: "500mg",
    route: "oral",
    scheduledAt: "2026-09-03T10:00"
  };

  const result = validateMedicationInput(input);

  expect(result).toBe("O campo Nome do medicamento é obrigatório");
});
```

CT03: Verifica a rejeição de dosage vazio, com os campos anteriores válidos. Atende a nós, arestas e caminho.

```
test("CT03 - rejeita dosagem em branco", () => {
  const input = {
    patientName: "Jorge",
    medicationName: "Dipirona",
    dosage: "",
    route: "oral",
    scheduledAt: "2026-09-03T10:00"
  };

  const result = validateMedicationInput(input);

  expect(result).toBe("O campo Dosagem é obrigatória");
});
```

CT04: Verifica a rejeição de route vazio, após a validação dos campos anteriores. Atende a nós, arestas e caminho.

```
test("CT04 - rejeita via de administração em branco", () => {
  const input = {
    patientName: "Jorge",
    medicationName: "Dipirona",
    dosage: "500mg",
    route: "",
    scheduledAt: "2026-09-03T10:00"
  };

  const result = validateMedicationInput(input);

  expect(result).toBe("O campo Via de administração é obrigatória");
});
```

CT05: Verifica a rejeição de scheduledAt vazio, exercitando a primeira condição da validação composta. Atende a nós, arestas, condição composta e caminho.

```
test("CT05 - rejeita data/hora em branco", () => {
  const input = {
    patientName: "Jorge",
    medicationName: "Dipirona",
    dosage: "500mg",
    route: "Oral",
    scheduledAt: ""
  };

  const result = validateMedicationInput(input);

  expect(result).toBe("O campo Data e horário são obrigatórios e deve estar no formato AAAA-MM-DD.");
});
```

CT06: Verifica a rejeição de scheduledAt com formato inválido, exercitando a segunda condição da validação composta. Atende a nós, arestas, condição composta e caminho.

```
test("CT06 - rejeita formato de data inválido", () => {
  const input = {
    patientName: "Jorge",
    medicationName: "Dipirona",
    dosage: "500mg",
    route: "Oral",
    scheduledAt: "2026-09-03T:00"
  };

  const result = validateMedicationInput(input);

  expect(result).toBe("O campo Data e horário são obrigatórios e deve estar no formato AAAA-MM-DD.");
});
```

CT07: Verifica o cenário em que todos os campos são válidos, percorrendo as saídas falsas das decisões até o return null. Atende a nós, arestas, condição composta e caminho.

```
test("CT07 - valida corretamente todos os campos", () => {
  const input = {
    patientName: "Jorge",
    medicationName: "Dipirona",
    dosage: "500mg",
    route: "Oral",
    scheduledAt: "2026-09-03T10:00"
  };

  const result = validateMedicationInput(input);

  expect(result).toBe(null);
});
```

7. Análise dos resultados

Após a execução dos testes, todos os 7 casos de teste passaram sem problemas, confirmando que a função `validateMedicationInput` apresentou o comportamento esperado nos cenários planejados. O relatório de cobertura gerado pelo Vitest apresentou 100% de cobertura de linhas e 100% de cobertura de ramos na função analisada.

A cobertura dos nós e das arestas do Grafo de Fluxo de Controle foi verificada por meio da correspondência entre os casos de teste e os caminhos básicos definidos anteriormente. Cada caso de teste está associado a um caminho específico, do CT-01/CB-01 até o CT-07/CB-07, contemplando as saídas verdadeira e falsa das decisões previstas no GFC.

Dessa forma, os 7 caminhos básicos, definidos a partir da complexidade ciclomática $V(G) = 7$, foram exercitados. Também não foram identificadas linhas sem cobertura nos cenários planejados, conforme indicado pela coluna “Uncovered Line #s” vazia no relatório do Vitest.

```
RUN v4.1.11 C:/Users/Kamila/Documents/ADS - 25/4 semestre/Engenharia de Software III/Trabalho 1 - testes
Coverage enabled with v8

✓ src/validateMedicationInput.test.ts (7 tests) 6ms
✓ CT01 - rejeita paciente em branco 3ms
✓ CT02 - rejeita medicamento em branco 0ms
✓ CT03 - rejeita dosagem em branco 0ms
✓ CT04 - rejeita via de administração em branco 0ms
✓ CT05 - rejeita data/hora em branco 0ms
✓ CT06 - rejeita formato de data inválido 0ms
✓ CT07 - valida corretamente todos os campos 0ms

Test Files 1 passed (1)
Tests 7 passed (7)
Start at 10:55:46
Duration 348ms (transform 32ms, setup 0ms, import 53ms, tests 6ms, environment 0ms)

% Coverage report from v8
-----|-----|-----|-----|-----|-----
File                                         | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----
All files                                   | 100      | 100      | 100      | 100      |
validateMedicationInput.ts                 | 100      | 100      | 100      | 100      |
-----|-----|-----|-----|-----|-----
```

Quanto à complexidade ciclomática, o resultado obtido por meio da ferramenta ESLint, utilizando a regra `complexity`, foi 7, coincidindo com o valor obtido manualmente pelas duas fórmulas utilizadas. Dessa forma, não houve divergência entre a análise manual e o resultado apresentado pela ferramenta.

```
7:8 error Function 'validateMedicationInput' has a complexity of 7. Maximum allowed is 2 complexity

✖ 1 problem (1 error, 0 warnings)
```

Os testes realizados não indicaram falhas no comportamento esperado da função nos cenários analisados. Como possível melhoria, as mensagens de erro poderiam ser estruturadas utilizando códigos e textos separados, o que facilitaria futuras alterações e a manutenção do código.

8. Repositório da atividade prática realizada

Link gitHub: [Atividade pratica - tecnica caixa branca](#)